

Plan: Gömb-strict Bitcoin architectural levers

2026-05-14

1 Hypothesis

The current Gömb-strict numbers on Bitcoin (0.8972 ± 0.0079 Alpha, 0.9145 ± 0.0068 OTC) sit ~ 0.03 AUC below published transductive baselines (SGCN: $0.929/0.942$). The configs were inherited from a prior joint_mix Optuna sweep that targeted a different protocol; they are not architecturally optimised for strict-protocol Bitcoin.

We hypothesise that one or more single-knob changes from the v5_combined baseline can lift Bitcoin strict above the published transductive numbers, on the same consumer hardware. No Optuna — manual lever picks based on the dataset’s small-and-dense regime.

2 Levers (each from the v5_combined baseline)

Label	Change vs v5_combined	Why
L1 c5_arity	cycle_ks = c3+c4+c5+w2+w3	Bitcoin has many 5-cycles
L2 deeper	n_tiers = 3 $\rightarrow$ 5	deeper CPML routing
L3 wider_outer	M_outer = 8 $\rightarrow$ 16	more Clifford-FIR kernels
L4 long_train	n_epochs = 80 $\rightarrow$ 200	Bitcoin is small; can afford
L5 big_embed	d_embed = 32 $\rightarrow$ 64	small graph, big embedding fits

Each lever is independent and additive — if multiple lift the number, a follow-up phase can combine them (no Optuna, just stacking the levers that pass the test).

3 Scope and goal

5 levers $\times$ 2 datasets (Bitcoin Alpha, Bitcoin OTC) $\times$ 5 seeds = 50 runs. At ~ 30 s/seed on the RTX 2070 SUPER, total wall time ~ 25 min plus overhead ≈ 1 h.

Goal: identify the strongest single lever (or two) on Bitcoin strict. Then a follow-up phase can stack them.

4 Affected files

- signedkan_wip/experiments/run_gomb_bitcoin_levers_2026_05_14.sh — new runner script.
- No source edits. The runner just varies CLI flags on the existing run_gomb_smoke.py entry point.

5 CORE.YAML items touched

None.

6 Test strategy

This is an experiment, not a code change. The protocol is the same strict-protocol 5-seed evaluation used in the Gömb-strict main benchmark (`run_gomb_strict_benchmark_2026_05_14.sh`).

7 Performance budget

- Total wall: ≤ 90 minutes.
- Per-seed wall: ≤ 120 s (was 30 s for `v5_combined`; `long_train` lever extends to ~ 75 s).
- Peak RSS: ≤ 8 GB (Bitcoin is small).
- GPU memory: ≤ 6 GB on RTX 2070 SUPER.

8 Rollback path

The runner script is independent of any source. To rollback: delete the script. No source files touched.

9 Risk anticipation

- *No-lift outcome.* If every lever fails to beat SGCN transductive on Bitcoin, the honest framing (“strict-protocol penalty is real on Bitcoin”) stands; the experiment is documented as a negative result.
- *Single-lever lift.* If one lever wins, a follow-up phase runs a 5-seed paired comparison with the `v5_combined` baseline on the same seeds.
- *Multi-lever interaction.* If two or more lift independently, a follow-up phase stacks them; no Optuna.

10 Decision tree

- Any lever lifts Alpha *and* OTC by $\geq +0.02$ AUC $\Rightarrow$ promote to a 5-seed paired test against `v5_combined`; update the headline tables.
- All levers $< +0.01$ AUC $\Rightarrow$ document as a negative-result anchor; strict penalty is real on this dataset regime.
- Some levers help one dataset but not the other $\Rightarrow$ document dataset-specific configs in the paper’s “configuration policy” section.

11 Bottom line

5 manual lever picks, 50 runs, ~1 hour total. No Optuna. The hypothesis is that Gömb's architectural capacity exceeds what the current Bitcoin config exploits, and that strict-protocol Bitcoin can match or exceed published transductive numbers without going down the hyperparameter-tuning rabbit hole.